

Let us execute the `hostname` command inside a running container. But before that, let us find the container name, as follows:

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND
CREATED       STATUS        PORTS
NAMES
905464421cb3   alpine        "tail"
About a minute ago Up About a minute
pedantic_kepler
```

In my case, the container name is *pedantic_kepler*. Now let us execute the `hostname` command inside it, as follows:

```
$ docker exec pedantic_kepler hostname
905464421cb3
```

The above output shows the hostname of a container which is the same as a container ID.

This approach is good when you want to execute a single command. But what if you want to execute multiple commands? In that case, you can create an interactive session with a container. For instance, the command below creates an interactive session with a container using the `sh` shell:

```
$ docker exec -it pedantic_kepler /bin/sh
/ # hostname
905464421cb3
/ #
/ # uname -a
Linux 905464421cb3 4.18.0-10-generic #11-Ubuntu SMP Thu Oct
11 15:13:55 UTC 2018 x86_64 Linux
/ # exit
```

 **Note:** Type `Exit` or press ‘`Ctrl + D`’ to end the interactive session with a container.

In the above command:

- a) `-i` option keeps STDIN open even if it is not attached.
- b) `-t` option allocates a pseudo-TTY.

2. *docker cp*

We often need to copy files inside a container from the host machine and vice versa. We can do that easily with the *docker cp* command.

First, let us look at how to copy a file from the host machine to a Docker container. Shown below is the syntax for the same:

```
docker cp [OPTIONS] <SRC_PATH> <CONTAINER>:<DEST_PATH>
```

Now let us copy the source file—*file-from-host.sh*’ inside ‘*pedantic_kepler*’ container, as follows:

```
$ docker cp file-from-host.sh pedantic_kepler:/tmp
```

Next, let us verify whether the file has indeed been copied:

```
$ docker exec pedantic_kepler ls -lrth /tmp
total 0
-rw-r--r-- 1 0 0 20 Dec 31 16:34 file-from-host.sh
```

Yes, it’s there.

Now, let us look at how to copy a file from a container to the host machine. The syntax for this command is as follows:

```
docker cp [OPTIONS] <CONTAINER>:<SRC_PATH> <DEST_PATH>
```

Next, copy the *file-from-host.sh* file from the *pedantic_kepler* container to the host machine as follows:

```
$ docker cp pedantic_kepler:/tmp/file-from-host.sh ${PWD}
```

Let us verify that the file has indeed been copied.

```
$ ls -lrth
total 0
-rw-r--r-- 1 jarvis jarvis 20 Dec 31 22:04 file-from-host.sh
```

Yes, it’s been copied.

3. *docker inspect*

The *docker inspect* command provides low-level information on Docker objects. For example, if we execute this command on a container, then it will provide useful information like its ID, status, image used to spawn the container, network related information, and so on. We can use this command with various Docker objects like image, container, volume, etc.

Let us execute this command on a running container. Shown below is the output for the same:

```
$ docker inspect pedantic_kepler
[
  {
    "Id": "905464421cb38c84 f30b91febad1ef9
8026564fb2b7ada310480996a2a844081",
    "Created": "2019-12-31T15:58:16.328534109Z",
    "Path": "tail",
    "Args": [],
    "State": {
      "Status": "running",
      "Running": true,
```